

Correctifs de sécurité recommandés

1. Révoquer et remplacer immédiatement les secrets

Les secrets suivants doivent être régénérés :

- JWT_SECRET
- MONGO_URI
- RESEND_API_KEY
- FEDAPAY_SECRET_KEY
- FEDAPAY_WEBHOOK_SECRET
- CLOUDINARY_API_SECRET
- EMAIL_PASS

Pourquoi :

- ils sont présents dans le fichier .env local
- même s'il est ignoré par Git, il peut être copié, sauvegardé ou exposé localement
- un secret compromis peut donner accès à la base, au paiement et aux mails

Action :

- générer de nouvelles clés
- les remplacer dans l'environnement de production
- les stocker via un gestionnaire de secrets ou variables d'environnement sécurisées

2. Renforcer la validation des webhooks de paiement

Fichier concerné : services/webhookService.js

À faire :

- forcer la vérification de signature en production
- refuser toute requête webhook non signée
- ne pas autoriser de fallback unsigned quand le secret est absent

- journaliser toutes les tentatives invalides
- alerter sur les anomalies

Risque :

- une commande peut être créée sans paiement réel

3. Sécuriser les JWT et la session

Fichiers concernés :

- Middlewares/authMiddleware.js
- Middlewares/verifyTokens.js
- utils/socket.js

À faire :

- utiliser un JWT_SECRET fort et unique
- utiliser des durées d'expiration courtes
- ajouter un mécanisme de refresh token si nécessaire
- vérifier les rôles à chaque route sensible
- empêcher les tokens expirés ou modifiés

4. Renforcer les permissions admin

Fichiers concernés :

- routes/adminRoutes.js
- routes/authRoutes.js

À faire :

- vérifier le rôle admin à chaque accès critique
- contrôler les droits sur les ressources utilisateur
- ajouter une journalisation d'audit des actions super admin
- sécuriser les routes de création, suppression et modification

5. Empêcher les fuites d'erreurs côté API

Fichier concerné : server.js

À faire :

- ne pas exposer les messages internes en production
- ne pas renvoyer `error.message` dans les réponses API
- logger les erreurs côté serveur sans exposer la stack

6. Restreindre les origines CORS

Fichiers concernés :

- server.js
- utils/socket.js

À faire :

- autoriser uniquement les domaines connus
- séparer les environnements dev/staging/prod
- supprimer les origines non nécessaires
- contrôler correctement les credentials

7. Renforcer la protection contre les abus

À conserver et améliorer :

- rate limiting déjà présent dans server.js
- protections sur les logins et OTP

À ajouter :

- limite sur les routes de paiement, reset password, upload
- blocage de brute force sur les comptes admin
- validation stricte des fichiers uploadés

8. Ajouter un fichier .env.example sans secrets réels

À créer :

- .env.example

Il doit contenir :

- le nom des variables
- des valeurs factices ou placeholders
- aucune clé réelle

9. Ajouter une vérification CI pour les secrets

À mettre en place :

- scanner le dépôt pour détecter les secrets
- bloquer les commits contenant des clés sensibles
- ajouter une validation dans le pipeline CI/CD

Priorité des actions

Priorité 1 - à faire tout de suite

- rotation des secrets
- validation webhook en production
- sécurisation JWT
- contrôle des routes admin

Priorité 2 - sous 1 à 2 jours

- logs d'audit
- CORS strict
- validation upload

- .env.example

Priorité 3 - dans la semaine

- CI secret scanning

- 2FA admin

- revue complète des endpoints sensibles

Conclusion

Le correctif principal est de :

1. remplacer les secrets
2. imposer la validation des webhooks en production
3. renforcer l'authentification et les permissions admin
4. sécuriser les routes et les données sensibles

Cela permettra d'éliminer les risques critiques les plus importants avant la mise en production.